

Applied Deep Learning

Chapter 2: Feedforward Neural Networks

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering
University of Toronto

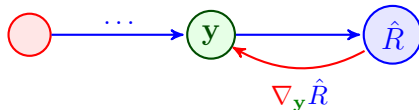
Winter 2026

Backpropagation: Local Operations

Let's consider a general problem: an *objective scalar* $\hat{R}$ is a function of *K -dimensional vector* $\mathbf{y} \in \mathbb{R}^K$. Clearly in this case, we have a *gradient*

$$\nabla_{\mathbf{y}} \hat{R} = \begin{bmatrix} \partial \hat{R} / \partial y_1 \\ \vdots \\ \partial \hat{R} / \partial y_K \end{bmatrix}$$

Assume that we know this gradient. The *vector* $\mathbf{y}$ is also function of *another variable*. We want to compute *gradient of* $\hat{R}$ with respect to this *other variable*

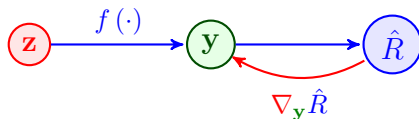


We now consider different cases for the *other variable* and its link to $\mathbf{y}$

Backpropagation: *Local Operation 1*

Entry-wise Functional Operation

$\mathbf{y} \in \mathbb{R}^K$ is a function of $\mathbf{z} \in \mathbb{R}^K$ as $\mathbf{y} = f(\mathbf{z})$ with $f(\cdot)$ operating entry-wise



For this case, we note that y_k is only a function of z_k ; thus we have

$$\frac{\partial \hat{R}}{\partial z_k} = \frac{\partial \hat{R}}{\partial y_k} \frac{\partial y_k}{\partial z_k} = \frac{\partial \hat{R}}{\partial y_k} \dot{f}(z_k)$$

So, we can use *entry-wise product* $\odot$ to get from $\nabla_{\mathbf{y}} \hat{R}$ to $\nabla_{\mathbf{z}} \hat{R}$

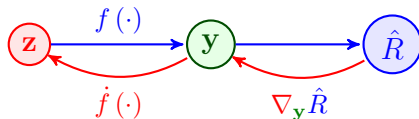
$$\nabla_{\mathbf{z}} \hat{R} = \nabla_{\mathbf{y}} \hat{R} \odot \dot{f}(\mathbf{z})$$

Backpropagation: Local Operation 1

Reminder: Entry-wise product of two vectors of the same size is

$$\mathbf{z} \odot \mathbf{y} \begin{bmatrix} z_1 \\ \vdots \\ z_K \end{bmatrix} \odot \begin{bmatrix} y_1 \\ \vdots \\ y_K \end{bmatrix} = \begin{bmatrix} y_1 z_1 \\ \vdots \\ y_K z_K \end{bmatrix}$$

So, we can compactly perform this local operation as follows



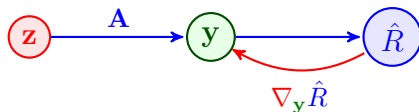
with the backward step

$$\nabla_{\mathbf{z}} \hat{R} = \nabla_{\mathbf{y}} \hat{R} \odot \dot{f}(\mathbf{z})$$

Backpropagation: Local Operation 2

Linear Vector-to-Vector Operation

$\mathbf{y} \in \mathbb{R}^K$ is a function of $\mathbf{z} \in \mathbb{R}^N$ as $\mathbf{y} = \mathbf{A}\mathbf{z}$ with $\mathbf{A} \in \mathbb{R}^{K \times N}$



Here, y_k is a linear function of $z_1, \dots, z_N$

$$y_k = \sum_{n=1}^N \mathbf{A}[k, n] z_n$$

where $\mathbf{A}[k, n]$ is entry of $\mathbf{A}$ at row k and column n . We thus can write

$$\frac{\partial \hat{R}}{\partial z_n} = \sum_{k=1}^K \frac{\partial \hat{R}}{\partial y_k} \frac{\partial y_k}{\partial z_n} = \sum_{k=1}^K \frac{\partial \hat{R}}{\partial y_k} \mathbf{A}[k, n]$$

Backpropagation: *Local Operation 2*

Let's denote column n of $\mathbf{A}$ by notation $\mathbf{A}[:, n]$; so, we can write

$$\frac{\partial \hat{R}}{\partial z_n} = \sum_{k=1}^K \frac{\partial \hat{R}}{\partial y_k} \mathbf{A}[k, n] = \mathbf{A}[:, n]^T \nabla_{\mathbf{y}} \hat{R}$$

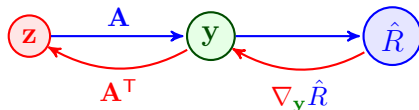
Now, if we collect them in a vector form we get

$$\nabla_{\mathbf{z}} \hat{R} = \begin{bmatrix} \partial \hat{R} / \partial z_1 \\ \vdots \\ \partial \hat{R} / \partial z_N \end{bmatrix} = \begin{bmatrix} \mathbf{A}[:, 1]^T \\ \vdots \\ \mathbf{A}[:, N]^T \end{bmatrix} \nabla_{\mathbf{y}} \hat{R} = \mathbf{A}^T \nabla_{\mathbf{y}} \hat{R}$$

This makes sense! Since we are *changing dimensions from K to N* , we need a product that *does such dimensionality change* for us

Backpropagation: *Local Operation 2*

Long story short . . .



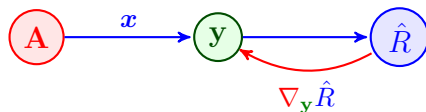
with backward step

$$\nabla_z \hat{R} = A^T \nabla_y \hat{R}$$

Backpropagation: Local Operation 3

Linear Matrix-to-Vector Operation

$\mathbf{y} \in \mathbb{R}^K$ is a function of $\mathbf{A} \in \mathbb{R}^{K \times N}$ as $\mathbf{y} = \mathbf{A}\mathbf{x}$ with $\mathbf{x} \in \mathbb{R}^N$



- + Wait a moment! The *other variable* is a matrix! How do we define $\nabla_{\mathbf{A}} \hat{R}$?
- Right! Let's first extend the definition

Backpropagation: Local Operation 3

Assume scalar $\hat{R}$ is a function of matrix $\mathbf{A} \in \mathbb{R}^{K \times N}$, we define

$$\nabla_{\mathbf{A}} \hat{R} = \begin{bmatrix} \partial \hat{R} / \partial \mathbf{A} [1, 1] & \dots & \partial \hat{R} / \partial \mathbf{A} [1, N] \\ \vdots & & \vdots \\ \partial \hat{R} / \partial \mathbf{A} [K, 1] & \dots & \partial \hat{R} / \partial \mathbf{A} [K, N] \end{bmatrix}$$

with $\mathbf{A} [k, n]$ being the entry of $\mathbf{A}$ at row k and column n

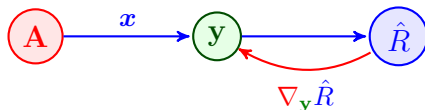
It is worth to also think of **gradient descent** in this case: assume we are minimizing $\hat{R}$ over $\mathbf{A}$ using **gradient descent** with **learning rate** η . At iteration t we got point $\mathbf{A}^{(t)}$; now, in the next iteration we can readily write

$$\mathbf{A}^{(t+1)} = \mathbf{A}^{(t)} - \nabla_{\mathbf{A}} \hat{R} |_{\mathbf{A}=\mathbf{A}^{(t)}}$$

so apparently everything is as before!

Backpropagation: *Local Operation 3*

Back to our problem, we can write



Entry k of $\mathbf{y}$ is a linear function of the k -th row of $\mathbf{A}$, i.e.,

$$y_k = \sum_{n=1}^N x_n \mathbf{A}[k, n]$$

So, we can write

$$\frac{\partial \hat{R}}{\partial \mathbf{A}[j, n]} = \sum_{k=1}^K \frac{\partial \hat{R}}{\partial y_k} \frac{\partial y_k}{\partial \mathbf{A}[j, n]} = \sum_{k=1}^K \frac{\partial \hat{R}}{\partial y_k} \mathbf{1}\{k = j\} x_n = \frac{\partial \hat{R}}{\partial y_j} x_n$$

Backpropagation: *Local Operation 3*

Let's now put them in a matrix

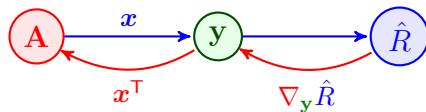
$$\nabla_{\mathbf{A}} \hat{R} = \begin{bmatrix} \frac{\partial \hat{R}}{\partial y_1} x_1 & \dots & \frac{\partial \hat{R}}{\partial y_1} x_N \\ \vdots & & \vdots \\ \frac{\partial \hat{R}}{\partial y_K} x_1 & \dots & \frac{\partial \hat{R}}{\partial y_K} x_N \end{bmatrix} = \nabla_{\mathbf{y}} \hat{R} \mathbf{x}^\top$$

So, we should now apply outer product! This again makes sense!

We have a K -dimensional gradient $\nabla_{\mathbf{y}} \hat{R}$ and an N -dimensional vector $\mathbf{x}$, we need an outer product to convert it into the $K \times N$ matrix $\nabla_{\mathbf{A}} \hat{R}$

Backpropagation: *Local Operation 3*

So, we could conclude



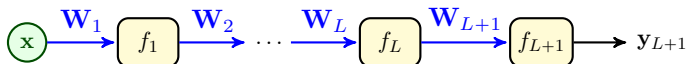
with the backward step

$$\nabla_{\mathbf{A}} \hat{R} = \nabla_{\mathbf{y}} \hat{R} \mathbf{x}^T$$

Now, we are ready to “*backpropagate*” over an FNN

Backpropagation: Algorithm

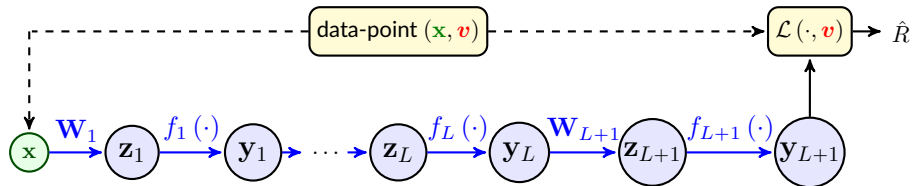
Let's recall the compact diagram of an FNN with L hidden layers



We can easily expand it into a *computation graph*

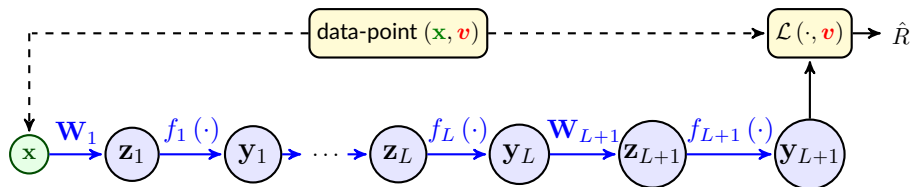


Our objective is *the empirical risk*; so let's include it also in the graph



Backpropagation: Algorithm

Given data-point $\mathbf{x}$ and its true label $\mathbf{v}$, we once complete a forward pass



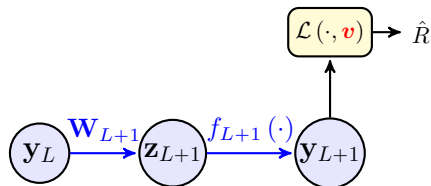
At the end of forward pass,

we know the value of all variables, i.e., $\mathbf{z}_\ell$ and $\mathbf{y}_\ell$ for all ℓ

Now, let's assume we want to find $\nabla_{\mathbf{W}_{L+1}} \hat{R}$

Backpropagation: Algorithm

We now cut the graph at the link $\mathbf{W}_{L+1}$



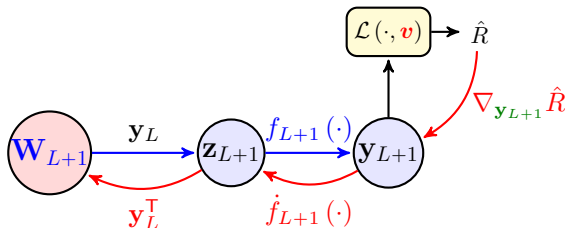
Let's recall ...

- + *what is the variable here?*
- It's $\mathbf{W}_{L+1}$
- + *Can we modify the graph such that it becomes a node?*
- Sure! We note that $\mathbf{z}_{L+1} = \mathbf{W}_{L+1}\mathbf{y}_L$. We can look at it as a linear *matrix*-to-vector operation; so, we could modify the graph as

Backpropagation: Algorithm

Let's now move backward to $\mathbf{W}_{L+1}$

- 1 We have $\mathbf{y}_{L+1}$, so we compute $\nabla_{\mathbf{y}_{L+1}} \hat{R}$, and pass it
- 2 We have $\mathbf{z}_{L+1}$, so we compute $\dot{f}_{L+1}(\mathbf{z}_{L+1})$, and then we get $\nabla_{\mathbf{z}_{L+1}} \hat{R}$
- 3 We have $\mathbf{y}_L$, so we compute $\nabla_{\mathbf{W}_{L+1}} \hat{R}$ from the last pass as $\nabla_{\mathbf{W}_{L+1}} \hat{R}$



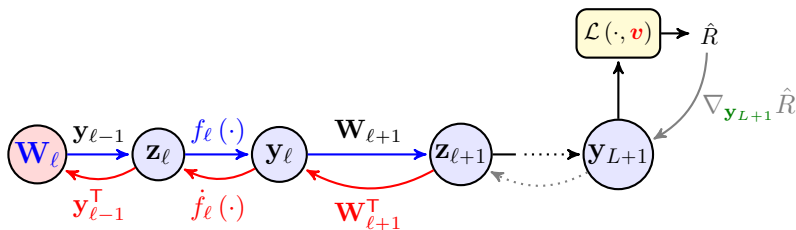
Backpropagation: Algorithm

We can *propagate* backward deeper and deeper

- We cut at the link that *we want to compute the gradient with respect to*
- We exchange the linear vector-to-vector function *at that particular link* to a linear *matrix*-to-vector function
- We move backwards till we get to the source of this graph

Backpropagation: Algorithm

As we arrive backward at layer ℓ , we already have messages till $\mathbf{z}_{\ell+1}$



Once we propagate back to the input, i.e., $\ell = 1$, then we have all gradients!

Backpropagation: *Few Notations*

To formally present backpropagation, let us *define a few notations*

For $\ell = 1, \dots, L + 1$, we define

$$\overleftarrow{\mathbf{y}}_{\ell} = \nabla_{\mathbf{y}_{\ell}} \hat{R}$$

$$\overleftarrow{\mathbf{z}}_{\ell} = \nabla_{\mathbf{z}_{\ell}} \hat{R}$$

and keep in mind that

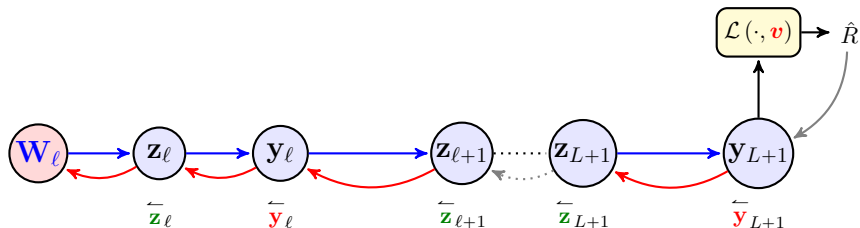
- $\mathbf{y}_{\ell}$ and $\overleftarrow{\mathbf{y}}_{\ell}$ are totally different things
- $\mathbf{z}_{\ell}$ and $\overleftarrow{\mathbf{z}}_{\ell}$ are totally different things

Backpropagation: Pseudo Code

```

1: Initiate with  $\bar{\mathbf{y}}_{L+1} = \nabla \mathcal{L}(\mathbf{y}_{L+1}, \mathbf{v})$  and  $\bar{\mathbf{z}}_{L+1} = \bar{\mathbf{y}}_{L+1} \odot \dot{f}_{L+1}(\mathbf{z}_{L+1})$ 
2: for  $\ell = L, \dots, 1$  do
3:   Determine  $\bar{\mathbf{y}}_{\ell} = \mathbf{W}_{\ell+1}^T \bar{\mathbf{z}}_{\ell+1}$  and drop  $\bar{\mathbf{y}}_{\ell}[0]$  # backward affine
4:   Determine  $\bar{\mathbf{z}}_{\ell} = \dot{f}_{\ell}(\mathbf{z}_{\ell}) \odot \bar{\mathbf{y}}_{\ell}$  # backward activation
5: end for
6: for  $\ell = 1, \dots, L+1$  do
7:   Return  $\nabla_{\mathbf{W}_{\ell}} \hat{R} = \bar{\mathbf{z}}_{\ell} \mathbf{y}_{\ell-1}^T$ 
8: end for

```



Backpropagation: Pseudo Code

```

1: Initiate with  $\overleftarrow{\mathbf{y}}_{L+1} = \nabla \mathcal{L}(\mathbf{y}_{L+1}, \mathbf{v})$  and  $\overleftarrow{\mathbf{z}}_{L+1} = \overleftarrow{\mathbf{y}}_{L+1} \odot \dot{f}_{L+1}(\mathbf{z}_{L+1})$ 
2: for  $\ell = L, \dots, 1$  do
3:   Determine  $\overleftarrow{\mathbf{y}}_{\ell} = \mathbf{W}_{\ell+1}^T \overleftarrow{\mathbf{z}}_{\ell+1}$  and drop  $\overleftarrow{\mathbf{y}}_{\ell}[0]$  # backward affine
4:   Determine  $\overleftarrow{\mathbf{z}}_{\ell} = \dot{f}_{\ell}(\mathbf{z}_{\ell}) \odot \overleftarrow{\mathbf{y}}_{\ell}$  # backward activation
5: end for
6: for  $\ell = 1, \dots, L + 1$  do
7:   Return  $\nabla_{\mathbf{w}_{\ell}} \hat{R} = \overleftarrow{\mathbf{z}}_{\ell} \mathbf{y}_{\ell-1}^T$ 
8: end for

```

- + This looks very similar to forward propagation! Right?!
- Yeah! Just we go *backward*! That's the whole point of backpropagation

You need to go once *forth* and then *back* to determine all gradients

Let's put them next to each other

Backpropagation: Pseudo Code

ForwardProp():

- 1: Initiate with $\mathbf{y}_0 = \mathbf{x}$
- 2: **for** $\ell = 0, \dots, L$ **do**
- 3: Add $\mathbf{y}_\ell[0] = 1$ and determine $\mathbf{z}_{\ell+1} = \mathbf{W}_{\ell+1}\mathbf{y}_\ell$ # forward affine
- 4: Determine $\mathbf{y}_{\ell+1} = f_{\ell+1}(\mathbf{z}_{\ell+1})$ # forward activation
- 5: **end for**
- 6: **for** $\ell = 1, \dots, L + 1$ **do**
- 7: Return $\mathbf{y}_\ell$ and $\mathbf{z}_\ell$
- 8: **end for**

BackProp():

- 1: Initiate with $\bar{\mathbf{y}}_{L+1} = \nabla \mathcal{L}(\mathbf{y}_{L+1}, \mathbf{v})$ and $\bar{\mathbf{z}}_{L+1} = \bar{\mathbf{y}}_{L+1} \odot \dot{f}_{L+1}(\mathbf{z}_{L+1})$
- 2: **for** $\ell = L, \dots, 1$ **do**
- 3: Determine $\bar{\mathbf{y}}_\ell = \mathbf{W}_{\ell+1}^\top \bar{\mathbf{z}}_{\ell+1}$ and drop $\bar{\mathbf{y}}_\ell[0]$ # backward affine
- 4: Determine $\bar{\mathbf{z}}_\ell = \dot{f}_\ell(\mathbf{z}_\ell) \odot \bar{\mathbf{y}}_\ell$ # backward activation
- 5: **end for**
- 6: **for** $\ell = 1, \dots, L + 1$ **do**
- 7: Return $\nabla_{\mathbf{w}_\ell} \hat{R} = \bar{\mathbf{z}}_\ell \mathbf{y}_{\ell-1}^\top$
- 8: **end for**

Complete Training Loop via *Gradient Descent*

- + Say we use *backpropagation*; then, how does *gradient descent* look?
- Well! We should go back and forth with all data-points

Say we have dataset

$$\mathbb{D} = \{(\mathbf{x}_b, \mathbf{v}_b) \text{ for } b = 1, \dots, B\}$$

GradientDescent() :

```

1: Initiate with some initial values  $\{\mathbf{W}_\ell^{(0)}\}$  and set a learning rate  $\eta$ 
2: while weights not converged do
3:   for  $b = 1, \dots, B$  do
4:     NN.values  $\leftarrow$  ForwardProp ( $\mathbf{x}_b, \{\mathbf{W}_\ell^{(t)}\}$ )           # forward
5:      $\{\nabla_{\mathbf{W}_\ell^{(t)}} \hat{R}_b\} \leftarrow$  BackProp ( $\mathbf{x}_b, \mathbf{v}_b, \{\mathbf{W}_\ell^{(t)}\}, \text{NN.values}$ )   # backward
6:   end for
7:   for  $\ell = 1, \dots, L + 1$  do
8:      $\mathbf{W}_\ell^{(t+1)} \leftarrow \mathbf{W}_\ell^{(t)} - \eta \text{ mean}(\nabla_{\mathbf{W}_\ell^{(t)}} \hat{R}_1, \dots, \nabla_{\mathbf{W}_\ell^{(t)}} \hat{R}_B)$ 
9:   end for
10: end while

```